

## Lab12.2\_Transformers\_TextClassification\_Varshitha\_2403A52397

### STEP 2 — Install & Import Libraries

```
!pip install -U transformers accelerate datasets
```

```
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.10.12/dist-packages (from accelerate==1.0.1)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.10.12/dist-packages (from accelerate==1.0.1)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.10.12/dist-packages (from accelerate==1.0.1)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.10.12/dist-packages (from accelerate==1.0.1)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.10.12/dist-packages (from accelerate==1.0.1)
```

```

Uninstalling datasets-4.0.0:
  Successfully uninstalled datasets-4.0.0
Attempting uninstall: transformers
  Found existing installation: transformers 5.0.0
Uninstalling transformers-5.0.0:
  Successfully uninstalled transformers-5.0.0
Successfully installed datasets-4.5.0 pyarrow-23.0.1 transformers-5.2.0

```

```
!pip install transformers datasets evaluate accelerate
```

```

Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12,
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.:
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/li
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12,
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/pytho
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib,
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/pytho
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/py
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/py
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/d:
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12,
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pyth
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dis
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/c
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/d:
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/pyt
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/d:
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.:
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python:
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.:

```

```
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist
Downloading evaluate-0.4.6-py3-none-any.whl (84 kB)
```

84.1/84.1 kB 2.5 MB/s eta 0:00:00

```
Installing collected packages: evaluate
```

```
Successfully installed evaluate-0.4.6
```

```
import numpy as np
import pandas as pd
import torch
import seaborn as sns
import matplotlib.pyplot as plt

from datasets import load_dataset
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments
)

from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

### Explanation (Text Cell)

Transformers library is used to load pretrained BERT models and tokenizers. Datasets library loads benchmark datasets. PyTorch is used for deep learning model training. Sklearn metrics are used for evaluation. Matplotlib and seaborn are used for visualization.

### STEP 3 — Load Dataset

```
dataset = load_dataset("imdb")
dataset
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Us
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings ta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access p
warnings.warn(
README.md:          7.81k/? [00:00<00:00, 149kB/s]

plain_text/train-00000-of-          21.0M/21.0M [00:00<00:00, 21.7MB/s]
00001.parquet: 100%

plain_text/test-00000-of-          20.5M/20.5M [00:00<00:00, 58.3MB/s]
00001.parquet: 100%

plain_text/unsupervised-          42.0M/42.0M [00:00<00:00, 24.9MB/s]
00000-of-00001.p(...): 100%

Generating train split: 100%          25000/25000 [00:00<00:00, 55934.07 examples/s]

Generating test split: 100%          25000/25000 [00:00<00:00, 138348.82 examples/s]

Generating unsupervised split: 100%          50000/50000 [00:00<00:00, 170876.20 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})

```

```
dataset['train'][0]
```

```

{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all
the controversy that surrounded it when it was first released in 1967. I
also heard that at first it was seized by U.S. customs if it ever tried to
enter this country, therefore being a fan of films considered
"controversial" I really had to see this for myself.<br /><br />The plot is
centered around a young Swedish drama student named Lena who wants to learn
everything she can about life. In particular she wants to focus her
attentions to making some sort of documentary on what the average Swede
thought about certain political issues such as the Vietnam War and race
issues in the United States. In between asking politicians and ordinary
denizens of Stockholm about their opinions on politics, she has sex with her
drama teacher, classmates, and married men.<br /><br />What kills me about I
AM CURIOUS-YELLOW is that 40 years ago, this was considered pornographic.
Really, the sex and nudity scenes are few and far between, even then it\'s
not shot like some cheaply made porno. While my countrymen mind find it
shocking, in reality sex and nudity are a major staple in Swedish cinema.

```

Even Ingmar Bergman, arguably their answer to good old boy John Ford, had sex scenes in his films.<br /><br />I do commend the filmmakers for the fact that any sex shown in the film is shown for artistic purposes rather than just to shock people and make money to be shown in pornographic theaters in America. I AM CURIOUS-YELLOW is a good film for anyone wanting to study the meat and potatoes (no pun intended) of Swedish cinema. But really, this film doesn\'t have much of a plot.',  
 'label': 0}

```
import pandas as pd

train_labels = dataset['train']['label']
pd.Series(train_labels).value_counts()
```

```

      count
0    12500
1    12500

dtype: int64
```

### Dataset Description (Copy)

The IMDb dataset is a binary sentiment classification dataset. It contains 50,000 movie reviews labeled as positive or negative. The dataset is evenly split between two classes. It is used for binary text classification tasks. The dataset is sourced from Hugging Face Datasets library.

### STEP 4 — Tokenization

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
tokenizer_config.json: 100%          48.0/48.0 [00:00<00:00, 5.70kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
vocab.txt: 100%                    232k/232k [00:00<00:00, 3.60MB/s]
tokenizer.json: 100%                466k/466k [00:00<00:00, 3.01MB/s]
```

```
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

|           |                                               |
|-----------|-----------------------------------------------|
| Map: 100% | 25000/25000 [00:27<00:00, 524.80 examples/s]  |
| Map: 100% | 25000/25000 [00:21<00:00, 1254.03 examples/s] |
| Map: 100% | 50000/50000 [00:44<00:00, 1029.23 examples/s] |

### Explanation (Text Cell)

Transformers use tokenization to convert text into subword tokens. Padding ensures equal sequence length. Truncation limits long sequences. This allows batch processing in neural networks.

### STEP 5 — Prepare Dataset for PyTorch

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format("torch", columns=["input_ids", "attention_mask"
```

### Explanation

Deep learning models require numerical tensor inputs. Text is converted into token IDs and attention masks. Labels are renamed to match model requirements.

### STEP 6 — Train-Test Split

```
train_dataset = tokenized_datasets["train"]
test_dataset = tokenized_datasets["test"]
```

### Explanation

The dataset already provides predefined training and testing splits. Separating training and testing data ensures fair model evaluation.

### STEP 7 — Load Pretrained BERT Model

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
config.json: 100% 570/570 [00:00<00:00, 38.5kB/s]
model.safetensors: 100% 440M/440M [00:05<00:00, 57.4MB/s]
Loading weights: 100% 199/199 [00:00<00:00, 600.24it/s, Materializing param=bert.pooler.dense
```

#### BertForSequenceClassification LOAD REPORT from: bert-base-uncased

| Key                                        | Status     |
|--------------------------------------------|------------|
| cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
| cls.seq_relationship.weight                | UNEXPECTED |
| cls.predictions.transform.dense.weight     | UNEXPECTED |
| cls.seq_relationship.bias                  | UNEXPECTED |
| cls.predictions.transform.dense.bias       | UNEXPECTED |
| cls.predictions.bias                       | UNEXPECTED |
| cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
| classifier.weight                          | MISSING    |
| classifier.bias                            | MISSING    |

#### Notes:

- *UNEXPECTED* :can be ignored when loading from different task/architecture
- *MISSING* :those params were newly initialized because missing from the che

## Explanation

Transfer learning uses a model pre-trained on large text corpora. Pretrained models improve performance and reduce training time. BERT captures contextual relationships between words.

## STEP 8 — Training Configuration

```
import transformers
print(transformers.__version__)
```

5.2.0

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100,
)
```

``logging_dir`` is deprecated and will be removed in v5.2. Please set ``TENSORBOARD``

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions)
    acc = accuracy_score(labels, predictions)
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
```

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions)
    acc = accuracy_score(labels, predictions)
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

```
import torch
print(torch.cuda.is_available())
```

True

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import numpy as np

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions)
    acc = accuracy_score(labels, predictions)
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
```



```
"f1": f1,  
}
```

```
from transformers import Trainer  
  
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=test_dataset,  
    compute_metrics=compute_metrics,  
)
```

```
trainer.train()
```

---

